

Corner refinement (Subpixel Corner Refinement)

PHỤ THUỘC A/05-lan-truyen-sai-so-pixel-sang-pose (σ là hệ số nhân của mọi công thức sai số — chương này là chương duy nhất quyết định nó); A/06-hieu-ung-vat-ly-va-quang-hoc (trần quang học — không phương pháp nào vượt qua); B/07-detection-pipeline (cơ chế định vị góc mặc định của từng detector).

TÓM TẮT

Chương này quyết định σ , và σ là đại lượng duy nhất trong A/05 mà phần mềm còn ảnh hưởng được — mọi thứ khác trong bốn công thức của chương ấy là hình học hoặc vật lý. Sáu phương pháp được bàn, xếp theo cơ chế chứ không theo tên: trực giao gradient (`cornerSubPix`), khớp bậc hai quanh điểm yên ngựa, khớp đường có trọng số rồi lấy giao, định vị cạnh bằng moment, khớp conic cho marker tròn, và căn chỉnh quang trắc toàn bộ mẫu. Hai kết luận đáng mang ra. Thứ nhất: *điểm yên ngựa chính xác hơn giao hai đường*, vì nó bị ràng buộc từ bốn hướng thay vì hai — và đó là cơ sở của target lai. Thứ hai, và nó rẻ đến mức thường bị bỏ qua: *trung bình N khung khi robot đứng yên giảm σ theo $1/\sqrt{N}$* , một cải thiện năm lần với ba mươi khung, không tốn gì ngoài một phần ba giây. Nếu chỉ nhớ một điều: *hãy làm cái rẻ trước* — trung bình nhiều khung trước khi đổi thuật toán, và đổi thuật toán trước khi đổi phần cứng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **độ chính xác dưới pixel (subpixel accuracy)** — khả năng định vị một đặc trưng với độ chính xác nhỏ hơn kích thước một pixel. Nó khả thi vì một cạnh đi qua giữa hai pixel để lại dấu vết trong *tỉ lệ độ sáng* của chúng — thông tin đó có thật và chỉ cần không vứt đi.
- **điểm yên ngựa (saddle point)** — điểm mà hàm độ sáng có cực đại theo một hướng và cực tiểu theo hướng vuông góc. Góc của một bàn cờ vua là một điểm như thế: đi dọc một đường chéo thì sáng lên, đi dọc đường chéo kia thì tối đi.
- **trực giao gradient** — tính chất: tại một góc lý tưởng, vectơ gradient của mọi pixel lân cận đều vuông góc với vectơ nối pixel đó tới góc. Lý do: gradient vuông góc với cạnh, và mọi cạnh đều đi qua góc. Đây là nguyên lý của `cornerSubPix` [1].
- **khớp quang trắc (photometric alignment)** — tối ưu trực tiếp trên *giá trị độ sáng* thay vì trên các đặc trưng đã trích xuất: tìm phép biến đổi làm ảnh mẫu khớp tốt nhất với vùng ảnh quan sát. Chính xác nhất nhưng chậm nhất.
- **hiệp phương sai của góc** — ma trận 2×2 mô tả bất định vị trí của một góc, và nó thường *dị hướng*: một góc trên một cạnh dài sắc nét được xác định tốt theo hướng vuông góc cạnh và kém hơn theo hướng dọc cạnh.
- **nhiều độc lập giữa các khung** — điều kiện để trung bình N khung giảm σ theo $1/\sqrt{N}$. Nó không luôn đúng: nhiễu do chiếu sáng nhấp nháy, do rung tuần hoàn, hoặc do thiên lệch cố định thì không độc lập, và trung bình không giúp gì.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn có ba lựa chọn cải thiện σ : đổi sang thuật toán tinh chỉnh tốt hơn, mua camera đắt hơn, hoặc dừng robot lại 0,3 giây và lấy trung bình. Xếp ba lựa chọn theo tỉ lệ lợi ích trên chi phí, và giải thích vì sao thứ tự ấy làm nhiều người bất ngờ.

- Q2. Góc bàn cờ vua và góc tag vuông đều là “một góc”. Vì sao định vị cái thứ nhất chính xác hơn, và câu trả lời nằm ở một con số đếm rất đơn giản?
- Q3. `cornerSubPix` dựa trên một tính chất hình học duy nhất của gradient tại một góc. Tính chất đó là gì, và điều kiện nào phải thoả để nó còn đúng?
- Q4. Bạn lấy trung bình 100 khung và σ chỉ giảm hai lần thay vì mười lần như lý thuyết. Nêu ba nguyên nhân hoàn toàn khác nhau, và một phép thử phân biệt chúng.
- Q5. Căn chỉnh quang trắc dùng *toàn bộ* pixel của tag thay vì chỉ vùng quanh bốn góc, nên về nguyên tắc nó phải chính xác hơn nhiều. Vì sao nó vẫn ít được dùng, và trong trường hợp nào thì nên dùng?

1.1 Đặt vấn đề

A/05 kết thúc với bốn công thức, và cả bốn đều có cùng một tử số: $Z\sigma$. Khoảng cách Z thì bài toán quyết định. Tiêu cự f , kích thước tag trên ảnh w , baseline B — ba mẫu số — là hình học, và chúng được quyết định ở B/09 bằng nhôm và vít. Còn lại đúng một đại lượng mà phần mềm ảnh hưởng được: σ .

Chương này là chương về σ .

Điều làm nó đáng dành một chương riêng — thay vì một mục trong B/07 — là một sự thật hơi phản trực giác: *độ chính xác dưới pixel không phải một mẹo mà là một nguyên lý*. Một cạnh đi qua giữa hai pixel không biến mất; nó để lại dấu vết trong tỉ lệ độ sáng của hai pixel ấy. Nếu pixel bên trái sáng 80% và pixel bên phải sáng 20%, thì cạnh nằm gần bên phải. Thông tin ấy có thật, và toàn bộ nghệ thuật của chương này là không vứt nó đi.

Bài toán gốc thuộc về ngành trắc lượng ảnh và thị giác máy tính từ những năm 1980, khi người ta nhận ra rằng độ chính xác của phép đo ảnh không bị giới hạn bởi kích thước pixel. Toán tử Förstner [1] là một trong những công trình sớm hình thức hoá chuyện này, và nguyên lý trực giao gradient của nó vẫn là nền của `cornerSubPix` trong OpenCV ngày nay.

Với bài toán docking, chương này có một đặc điểm đáng nói: *nó là chương có nhiều lựa chọn rẻ nhất*. Khác với B/09 — nơi mọi cải thiện đòi gia công lại — ở đây có những cải thiện chỉ tốn vài dòng code hoặc thậm chí không tốn gì. Và trong số đó, cái rẻ nhất lại là cái người ta hay nghĩ tới sau cùng: *lấy trung bình nhiều khung khi robot đứng yên*. Nó giảm σ theo $1/\sqrt{N}$, không đòi thuật toán mới, không đòi phần cứng mới, và nó tương thích với mọi phương pháp khác trong chương. Mục 5 dành cho nó, và tôi đặt nó trước các phương pháp tinh vi hơn một cách có chủ ý.

Cách hiển nhiên — và cách đúng — là dùng phương pháp tinh chỉnh tốt nhất mà detector cung cấp. Chỗ nó không đủ nằm ở hai điểm. Thứ nhất, các detector khác nhau dùng cơ chế khác nhau và các cơ chế ấy có trần khác nhau; biết cơ chế là biết trần. Thứ hai, có một trần cứng hơn nữa mà không phương pháp nào vượt qua được — trần quang học của A/06 mục 5 — và biết mình đang ở đâu so với trần ấy quyết định việc đầu tư thêm có đáng không.

TRỰC GIÁC

Vì sao đo được dưới pixel, nói bằng một hình ảnh.

Tưởng tượng một hàng rào có một vệt sơn trắng vắt ngang, và bạn phải nói vệt sơn bắt đầu ở thanh rào thứ mấy. Nếu mỗi thanh rào chỉ có hai trạng thái — có sơn hoặc không — thì bạn chỉ nói được “giữa thanh thứ 5 và thứ 6”. Độ chính xác bằng một thanh rào.

Nhưng nếu bạn nhìn được *mức độ* sơn trên từng thanh — thanh thứ 5 phủ hết, thanh thứ 6 phủ một phần ba, thanh thứ 7 sạch — thì bạn nói được chính xác hơn nhiều: mép vệt sơn

nằm ở một phần ba thanh thứ 6. Độ chính xác bây giờ là một phần ba thanh rào, và nếu bạn đọc được mức phủ chính xác hơn thì còn tốt hơn nữa.

Đó là toàn bộ nguyên lý. Pixel không phải ô vuông hai trạng thái; nó là một bộ đếm photon, và số photon nó đếm được tỉ lệ với *phần diện tích* của nó bị vùng sáng che phủ. Cạnh nằm ở đâu trong pixel thì tỉ lệ ấy nói ra.

Bây giờ đến phần thứ hai, và nó giải thích vì sao có nhiều phương pháp khác nhau. Một cạnh chỉ cho bạn biết vị trí theo *một* hướng — hướng vuông góc với nó. Để định vị một điểm trong mặt phẳng, cần ít nhất hai hướng.

Cách thứ nhất: lấy hai cạnh cắt nhau và tìm giao điểm. Cách này dùng hai hướng, và nó là cách của tag vuông. Chỗ yếu: nếu hai cạnh gần song song thì giao điểm trượt rất xa khi một trong hai cạnh nghiêng đi chút xíu.

Cách thứ hai: tìm một điểm mà độ sáng đổi chiều theo *mọi* hướng quanh nó — một điểm yên ngựa, như đỉnh của một cái yên. Điểm ấy bị ràng buộc từ bốn phía thay vì hai, nên nó không trượt được. Đây là cách của bàn cờ vua, và đó là lý do nó chính xác hơn.

1.2 Trực giao gradient: **cornerSubPix**

Phương pháp được dùng rộng rãi nhất, và nó trả lời Q3.

Nguyên lý dựa trên một quan sát hình học duy nhất: *tại một góc, vectơ gradient của mọi pixel lân cận đều vuông góc với vectơ nối pixel đó tới góc*. Lý do: gradient luôn vuông góc với cạnh (nó chỉ hướng độ sáng đổi nhanh nhất, tức hướng cắt ngang cạnh), và mọi cạnh tại một góc đều đi qua chính góc ấy. Nên nếu $\mathbf{q}$ là vị trí góc thật và $\mathbf{p}_i$ là một pixel lân cận với gradient ∇I_i , thì

$$\nabla I_i^\top (\mathbf{p}_i - \mathbf{q}) = 0.$$

Với nhiều pixel, đây là một hệ quá xác định; giải bình phương tối thiểu cho

$$\mathbf{q} = \left(\sum_i \nabla I_i \nabla I_i^\top \right)^{-1} \left(\sum_i \nabla I_i \nabla I_i^\top \mathbf{p}_i \right). \quad (1.1)$$

Đây là công thức của **cornerSubPix**, và nó bắt nguồn từ toán tử Förstner [1].

Ba điều đáng đọc ra từ (1.1), và cả ba đều có hệ quả thực hành.

Thứ nhất, ma trận $\sum \nabla I \nabla I^\top$ chính là ma trận cấu trúc, cùng ma trận xuất hiện trong bộ phát hiện góc Harris. Điều kiện để (1.1) có nghiệm tốt là ma trận ấy khả nghịch tốt, tức có hai giá trị riêng lớn — tức có gradient theo *hai* hướng khác nhau. Một điểm trên một cạnh thẳng chỉ có gradient theo một hướng, ma trận suy biến, và phương pháp không định vị được. Đây là câu trả lời cho phần thứ hai của Q3: *phải thật sự có một góc*, không phải một điểm trên cạnh.

Thứ hai, nghịch đảo của ma trận cấu trúc chính là hiệp phương sai (sai khác một hệ số phụ thuộc mức nhiễu ảnh). Nghĩa là phương pháp này cho *miễn phí* một ước lượng bất định cho từng góc, và nó dị hướng — đúng thứ mà A/03 mục 6 cần cho trọng số. Rất ít cài đặt lấy nó ra, và đó là một thiếu sót dễ sửa.

Thứ ba, kích thước của sổ lân cận là tham số quan trọng nhất, và nó là một đánh đổi thật. Cửa sổ lớn cho nhiều pixel hơn nên ít nhiễu hơn, nhưng nó có thể trùm sang các cạnh khác của tag hoặc sang vùng có nội dung khác, làm giả định trực giao bị vi phạm. Quy tắc thực hành: cửa sổ nên nhỏ hơn khoảng cách tới góc gần nhất, và với tag thì thường là vài pixel.

1.3 Điểm yên ngựa: vì sao chessboard thẳng

Mục này trả lời Q2, và nó là mục quan trọng nhất về mặt kết luận thiết kế.

Góc của một bàn cờ vua có một cấu trúc đặc biệt: hai ô đen và hai ô trắng gặp nhau tại một điểm, xếp chéo nhau. Hàm độ sáng quanh điểm ấy, khi khớp bằng một đa thức bậc hai

$$I(x, y) \approx a_0 + a_1x + a_2y + a_3x^2 + a_4xy + a_5y^2,$$

có một điểm dừng mà Hessian *không xác định dấu* — cực đại theo một hướng, cực tiểu theo hướng vuông góc. Đó là điểm yên ngựa, và vị trí của nó tìm được bằng cách cho gradient của đa thức bằng không:

$$\begin{pmatrix} 2a_3 & a_4 \\ a_4 & 2a_5 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = - \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}.$$

Đây là phương pháp khớp bậc hai, và nó được dùng trong các cài đặt chính xác cao [2], [3].

Bây giờ đến câu hỏi vì sao nó chính xác hơn giao hai đường, và câu trả lời nằm ở một con số đếm — đúng như Q2 gợi ý.

Với *giao hai đường*: vị trí góc được xác định bởi hai ràng buộc, mỗi đường một ràng buộc. Sai số của mỗi đường lan vào giao điểm, và hệ số lan phụ thuộc góc giữa hai đường: khi chúng gần vuông góc thì hệ số cỡ 1, khi chúng gần song song thì hệ số lớn tùy ý. Với một tag nhìn rất chếch, hai cạnh kề có thể gặp nhau ở góc khá nhỏ, và khi ấy sai số bị khuếch đại mạnh.

Với *điểm yên ngựa*: vị trí được xác định bởi cấu trúc độ sáng theo *mọi* hướng quanh điểm. Bốn góc phần tư quanh nó — hai sáng, hai tối, xen kẽ — mỗi cái đóng góp một ràng buộc, và bốn ràng buộc ấy đến từ bốn hướng phân bố đều. Không có cấu hình nào làm hệ suy biến theo cách mà hai đường gần song song làm suy biến bài toán giao điểm.

Nói ngắn: *bốn hướng ràng buộc thay vì hai*, và bốn hướng ấy phân bố đều thay vì có thể chụm lại.

Ghi chú trung thực: tôi dẫn kết luận này bằng lập luận cấu trúc (của a). Tôi *chưa* có một so sánh định lượng trên cùng điều kiện đo (của b và c chưa qua). Sự thật rằng cộng đồng hiệu chuẩn camera dùng chessboard chứ không dùng tag vuông là một bằng chứng gián tiếp ủng hộ, nhưng nó không phải một phép đo. Đây là mệnh đề nền của đề xuất target lai, nên nó đáng kiểm — và mini project cung cấp cách kiểm.

CẦN NHỚ

Sự khác biệt giữa hai cơ chế dẫn thẳng tới một đề xuất thiết kế, và đó là một trong ba luận điểm “nói ngược thói quen” của 00-map.

Tag vuông (AprilTag, ArUco) giới *nhận diện*: đọc được từ xa, bền với che khuất và bẩn, tỉ lệ báo sai cực thấp, và tự nói cho bạn biết nó là ai.

Góc chessboard giới *đo*: bốn hướng ràng buộc, không có cấu hình suy biến, và không chịu bias kiểu ellipse của marker tròn.

Đề xuất: *target lai*. Tag vuông ở ngoại vi để bắt bám và định danh từ xa; một vùng ChArUco ở trung tâm để đo tinh ở cự ly gần. ChArUco đặc biệt tiện vì nó *đã là* một target lai — ArUco định danh, chessboard đo (B/07 mục 3B).

Điều cần nhớ: đây không phải chọn một trong hai. Hai cơ chế phục vụ hai giai đoạn khác nhau của cùng một hành trình tiếp cận, và chiến lược coarse-to-fine ở B/13 mục 5 chính là cái khung để dùng cả hai.

1.4 Các phương pháp còn lại

Bốn phương pháp nữa, ngắn gọn theo cơ chế.

Khớp đường có trọng số gradient rồi lấy giao. Đây là cơ chế của AprilTag 2/3, đã mô tả ở B/07 mục 2. Ưu điểm so với cornerSubPix: nó dùng *toàn bộ chiều dài cạnh* — hàng trăm pixel — thay vì một cửa sổ nhỏ quanh góc. Nhược điểm: nó chịu bài toán khuếch đại tại giao điểm, và nó đòi cạnh thật sự thẳng (A/01 mục 4).

Định vị cạnh bằng moment. Dùng các moment của profile độ sáng — thường là moment Zernike — để xác định vị trí cạnh dưới pixel một cách có cơ sở giải tích [4]. Chính xác và có lý thuyết đẹp; ít dùng trong thực hành marker vì nó đòi mô hình profile cạnh đúng, và profile thật bị ảnh hưởng bởi MTF của ống kính theo cách khó mô hình hoá.

Khớp conic và hiệu chỉnh tâm. Cho marker tròn: khớp một ellipse lên các điểm biên, rồi — và đây là bước bắt buộc — hiệu chỉnh bias tâm bằng đại số conic [5]. Độ chính xác định vị tâm rất cao vì tâm được xác định bởi hàng trăm điểm biên. Nhưng A/02 mục 6 đã cảnh báo: bỏ bước hiệu chỉnh thì bạn có một phép đo rất chính xác quanh một giá trị thiên lệch phụ thuộc tư thế, và với ngân sách 5 mm thì thiên lệch nguy hiểm hơn nhiều.

Căn chỉnh quang trắc. Thay vì trích xuất đặc trưng rồi khớp, tối ưu trực tiếp trên giá trị độ sáng: tìm phép biến đổi (homography) làm ảnh mẫu của tag khớp tốt nhất với vùng ảnh quan sát. Hai kỹ thuật chuẩn là ECC [6] — bất biến với thay đổi sáng tuyến tính — và inverse compositional alignment [7] — nhanh hơn nhờ tính trước Hessian.

Đây là câu trả lời cho Q5. Về nguyên tắc nó *phải* chính xác nhất, vì nó dùng mọi pixel của tag chứ không chỉ vùng quanh bốn góc — có thể là hàng nghìn pixel thay vì vài chục. Ba lý do nó vẫn ít được dùng: nó chậm hơn một bậc; nó cần khởi tạo tốt nên vẫn phải chạy detector thường trước; và nó nhạy với sai khác giữa mẫu và thực tế — tag bị bẩn, chiếu sáng không đều, hoặc mẫu không khớp chính xác với tag in ra đều làm nó lệch. Trường hợp nên dùng: khi robot đã dừng (nên chậm không sao), khi tag sạch và chiếu sáng kiểm soát được, và khi bạn cần vắt kiệt σ — tức chính xác pha cuối của B/13 mục 5.

Bảng 1.1. Sáu phương pháp: cơ chế, số hướng ràng buộc, và khi nào dùng.

Phương pháp	Cơ chế	Khi nào dùng
cornerSubPix [1]	Trực giao gradient trong cửa sổ nhỏ	Mặc định cho ArUco; nhớ bật, và nhớ lấy hiệp phương sai ra
Khớp bậc hai saddle [2], [3]	Bốn hướng ràng buộc	Chessboard/ChArUco/deltille ; chính xác nhất cho góc
Khớp đường có trọng số [8]	Hai đường, dùng cả chiều dài cạnh	Mặc định AprilTag; tốt nhưng khuếch đại khi cạnh chếch
Moment/Zernike [4]	Moment của profile cạnh	Ít dùng; đòi mô hình profile đúng
Conic + hiệu chỉnh tâm [5]	Khớp ellipse rồi bù bias	Marker tròn — và bước hiệu chỉnh là <i>bắt buộc</i>
Quang trắc (ECC, IC) [6], [7]	Tối ưu trên toàn bộ pixel	Pha cuối, robot đứng yên, cần vắt kiệt σ

1.5 Trung bình nhiều khung: cái rẻ nhất

Mục này trả lời Q1 và Q4, và tôi đặt nó riêng vì nó là kỹ thuật có tỉ lệ lợi ích trên chi phí cao nhất trong chương.

Nếu nhiều định vị góc độc lập giữa các khung và có trung bình không, thì trung bình N khung có độ lệch chuẩn

$$\sigma_N = \frac{\sigma}{\sqrt{N}}.$$

Với $N = 30$: cải thiện 5,5 lần. Với $N = 100$: cải thiện 10 lần. Chi phí: N chia cho tốc độ khung hình, tức với 30 fps thì 1 giây cho $N = 30$ — và trong thực tế thường ít hơn vì có thể lấy trung bình trong khi robot đang giảm tốc.

Đặt cạnh các lựa chọn khác, thứ tự ở Q1 trở nên rõ. Chuyển từ khớp đường sang saddle point cải thiện có lẽ hai tới ba lần. Mua camera đắt hơn cải thiện có lẽ hai lần và tốn tiền thật. Trung bình 30 khung cải thiện 5,5 lần và tốn một giây. Lý do thứ tự này làm nhiều người bất ngờ là vì nó không cảm thấy như “kỹ thuật” — nó không có thuật toán mới nào, không có gì để đọc paper về, và nó nghe như gian lận. Nhưng nó là cách sử dụng thông tin đúng đắn, và nó tương thích với mọi phương pháp khác trong chương.

A. Khi nào nó không hoạt động

Đây là phần thứ hai và quan trọng hơn, và nó trả lời Q4. Công thức $1/\sqrt{N}$ có một điều kiện: *nhiều độc lập giữa các khung và có trung bình không*. Ba cách vi phạm, và ba nguyên nhân của Q4.

Nguyên nhân một: có thành phần thiên lệch. Phần lớn các hiệu ứng ở A/06 — méo còn sót, bias tâm ellipse, motion blur nếu còn chuyển động — là thiên lệch chứ không phải nhiễu. Trung bình không chạm tới chúng. Triệu chứng: σ giảm theo $1/\sqrt{N}$ lúc đầu rồi *bão hoà* ở một sàn khác không.

Nguyên nhân hai: nhiễu tương quan theo thời gian. Chiếu sáng nhấp nháy (đèn huỳnh quang, đèn LED điều chế độ rộng xung), rung tuần hoàn của thân robot, hoặc nhiễu đọc có cấu trúc của cảm biến. Triệu chứng: σ giảm chậm hơn $1/\sqrt{N}$ ngay từ đầu, và tỉ lệ giảm phụ thuộc vào việc N khung trải qua bao nhiêu chu kỳ của hiện tượng tuần hoàn.

Nguyên nhân ba: lượng tử hoá. Nếu thuật toán tinh chỉnh của bạn có một bước lượng tử ẩn — chẳng hạn nó luôn trả về bội của $1/8$ pixel do cách nội suy — thì trung bình nhiều khung của cùng một giá trị lượng tử cho ra chính giá trị ấy. Triệu chứng: σ rất nhỏ nhưng giá trị trung bình nằm đúng trên một lưới.

Phép thử phân biệt ba nguyên nhân, và nó đơn giản: vẽ σ_N theo N trên trục $\log\text{--}\log$. Đường lý thuyết có độ dốc $-1/2$. Bão hoà ở một sàn nghĩa là nguyên nhân một. Độ dốc nhỏ hơn $1/2$ ngay từ đầu nghĩa là nguyên nhân hai. Giá trị trung bình rơi trên một lưới đều nghĩa là nguyên nhân ba. Đây là một trong những biểu đồ có giá trị chẩn đoán cao nhất mà bạn có thể vẽ, và nó gần như miễn phí.

CẦN NHỚ

Thứ tự đầu tư đúng để cải thiện σ , từ rẻ tới đắt:

Một, trung bình nhiều khung khi đứng yên — $1/\sqrt{N}$, không tốn gì ngoài thời gian, tương thích với mọi thứ khác.

Hai, bật tinh chỉnh góc nếu detector của bạn tắt nó mặc định — vài dòng cấu hình.

Ba, chuyển sang cơ chế saddle point cho khâu đo tinh (ChArUco) — thay đổi thiết kế target, nhưng không đổi phần cứng camera.

Bốn, cải thiện chiếu sáng (B/14) — rẻ hơn đổi camera, và nó cải thiện tính ổn định của σ chứ không chỉ giá trị của nó.

Năm, căn chỉnh quang trục cho pha cuối — tốn công cài đặt.

Sáu, đổi camera hoặc ống kính — đắt nhất, và chỉ đáng nếu bạn đã kiểm rằng quang học là nút thắt (A/06 mục 5B).

Và trước tất cả: vẽ đồ thị σ_N theo N . Nếu nó bão hoà sớm thì mọi đầu tư vào danh sách trên là vô ích — bạn đang có một thành phần thiên lệch, và bạn phải tìm nó trước.

1.6 Các con số: một cảnh báo cần lặp lại

Trong tài liệu về marker, người ta thường gặp các con số kiểu: góc AprilTag khoảng 0,1–0,3 px; góc ChArUco sau tinh chỉnh khoảng 0,03–0,1 px; tâm marker tròn có thể dưới 0,05 px. Chúng xuất hiện trong lộ trình gốc của cây này và tôi đã dùng 0,2 px làm giá trị mặc định xuyên suốt.

Cần nói rõ ba điều về các con số ấy.

Một, chúng là bậc độ lớn kinh nghiệm, không phải số đo có điều kiện xác định. Chúng không đi kèm thông tin về camera nào, ống kính nào, kích thước tag trên ảnh bao nhiêu, mức chiếu sáng ra sao. Một con số không có điều kiện đo là một con số vô nghĩa theo chuẩn của `learning-rules.md` mục 5, và tôi ghi rõ điều đó thay vì trình bày chúng như kết quả đo.

Hai, điều chúng nói đáng tin là thứ tự tương đối, không phải giá trị tuyệt đối. Rằng saddle point tốt hơn giao hai đường, và rằng tâm của một vùng lớn tốt hơn cả hai — ba điều ấy có cơ sở cấu trúc (mục 3 và 4) và chúng khớp với các con số. Rằng con số cụ thể là 0,03 hay 0,08 thì phụ thuộc hệ thống.

Ba, chênh lệch giữa chúng nhỏ hơn vẻ ngoài trong bức tranh tổng thể. Chuyển từ 0,2 px xuống 0,05 px là cải thiện bốn lần cho σ , và theo A/05 thì nó cải thiện mọi dòng ngân sách bốn lần — đáng kể. Nhưng nó không chạm tới các dòng hệ thống, và trong một hệ chưa đo constellation bằng BA thì các dòng ấy chiếm phần lớn ngân sách (B/09 mục 5). Nói cách khác: cải thiện σ bốn lần trên một ngân sách bị chi phối bởi sai số cơ khí thì gần như không đổi gì.

Có một công trình so sánh thực nghiệm nhiều họ tag trên cùng điều kiện [9], và nó là nguồn duy nhất trong cây có số đo trực tiếp so sánh được giữa các họ. Nếu bạn cần con số thì đó là nơi đọc — kèm theo việc đọc kỹ điều kiện đo của họ. Nhưng cách đúng nhất vẫn là đo trên chính hệ của bạn, và mini project của B/07 là cách làm.

1.7 Hiệp phương sai của góc: sản phẩm phụ đáng lấy

Mục ngắn nhưng nó nối chương này với A/03 mục 6, và nó là một cải thiện dễ mà ít ai làm.

Mọi phương pháp trong chương đều sinh ra một ước lượng bất định như sản phẩm phụ. Với `cornerSubPix`, đó là nghịch đảo của ma trận cấu trúc $\sum \nabla I \nabla I^T$ nhân với phương sai nhiễu ảnh. Với khớp bậc hai, đó là nghịch đảo Hessian của bài toán khớp. Với khớp đường rồi lấy giao, nó tính được bằng cách lan truyền hiệp phương sai của hai đường qua phép giao.

Điểm quan trọng: *hiệp phương sai ấy dị hướng*, và tính dị hướng mang thông tin thật. Một góc nằm trên một cạnh dài, sắc nét, có tương phản cao được xác định rất tốt theo hướng vuông góc cạnh và kém hơn nhiều theo hướng dọc cạnh. Một góc ở vùng bị loá có bất định lớn theo mọi hướng. Một góc của tag nhìn rất chếch có bất định kéo dài theo hướng mà hai cạnh gần song song.

Dùng thông tin ấy làm Σ_i trong PnP (A/03 mục 6) cho hai lợi ích. Lợi ích hiển nhiên: nghiệm chính xác hơn, vì các góc tốt được cho tiếng nói lớn hơn. Lợi ích ít hiển nhiên nhưng quan trọng hơn: *hiệp phương sai pose đầu ra trở nên trung thực*. Công thức $\mathcal{I}^{-1}$ của A/05 chỉ cho một ước lượng bất định đáng tin nếu Σ_i đầu vào đúng, và toàn bộ chuỗi phía sau — trọng số khi hợp nhất theo thời gian ở B/11, công chi-square ở B/09 mục 7, tiêu chí hội tụ ở B/13 mục 8 — phụ thuộc vào tính trung thực ấy.

Chi phí để lấy nó ra: vài dòng code, vì đại lượng đã được tính trong quá trình tinh chỉnh. Đây là một trong những cải thiện có tỉ lệ lợi ích trên công sức cao nhất trong cả Phần B, và nó bị bỏ qua gần như phổ biến.

1.8 Giới hạn và chế độ hỏng

Trần quang học. Không phương pháp nào vượt qua được giới hạn do MTF và nhiễu xạ đặt ra (A/06 mục 5). Nếu một cạnh sắc trải qua 5 pixel trên ảnh của bạn, thì hy vọng định vị tới 0,03 px là lạc quan bất kể thuật toán. Phép kiểm rẻ đã nêu ở A/06 mục 9: đo profile cạnh.

Mọi phương pháp giả định mô hình cạnh đúng. `cornerSubPix` giả định gradient trực giao với vectơ tới góc — sai khi cửa sổ trùm sang nội dung khác. Khớp bậc hai giả định profile độ sáng xấp xỉ được bằng đa thức bậc hai — sai khi tag rất nhỏ trên ảnh hoặc khi có loá. Khớp quang trắc giả định mẫu khớp với thực tế — sai khi tag bẩn.

Trung bình nhiều khung không chạm tới thiên lệch. Đã bàn kỹ ở mục 5A. Đây là giới hạn quan trọng nhất của kỹ thuật rẻ nhất, và đồ thị σ_N theo N là cách phát hiện.

Cải thiện σ chỉ ảnh hưởng tới các dòng ngẫu nhiên. Đã bàn ở mục 6. Trong một hệ chưa xử lý các nguồn hệ thống, chương này có đòn bẩy thấp. Thứ tự đúng: xử lý hệ thống trước (B/09 mục 5, B/12, hoặc B/13 mục 3), rồi mới tối ưu σ .

Mệnh đề “saddle point chính xác hơn” chưa được đo. Đã ghi rõ ở mục 3. Nó là mệnh đề nền cho target lai và nó đáng kiểm sớm.

1.9 Thực hành

Bốn việc, theo thứ tự.

Một, vẽ đồ thị σ_N theo N trên trục log-log trước khi làm bất cứ gì khác. Nó cho biết bạn đang bị giới hạn bởi nhiễu hay bởi thiên lệch, và câu trả lời quyết định toàn bộ chiến lược.

Hai, lấy hiệp phương sai từng góc ra và đưa vào PnP. Vài dòng code, lợi ích kép.

Ba, bật trung bình nhiều khung ở pha cuối — nó đi kèm tự nhiên với stop-and-go của B/13 mục 4.

Bốn, cân nhắc target lai nếu ngân sách góc chặt và các nguồn hệ thống đã được xử lý.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vì sao đo được dưới pixel	Pixel là bộ đếm photon, và số photon tỉ lệ với <i>phần diện tích</i> bị vùng sáng che phủ. Cạnh nằm đâu trong pixel thì tỉ lệ độ sáng nói ra. Thông tin có thật; nghệ thuật là không vớt nó đi.
••• Nguyên lý <code>cornerSubPix</code>	Tại góc thật, gradient của mọi pixel lân cận vuông góc với vectơ nối pixel đó tới góc (vì gradient vuông góc với cạnh, mà mọi cạnh đi qua góc) [1]. Giải bình phương tối thiểu hệ $\nabla I_i^T (\mathbf{p}_i - \mathbf{q}) = 0$.
••• Điều kiện để <code>cornerSubPix</code> hoạt động	Ma trận cấu trúc $\sum \nabla I \nabla I^T$ phải khả nghịch tốt, tức có gradient theo <i>hai</i> hướng khác nhau. Một điểm trên cạnh thẳng chỉ có một hướng $\Rightarrow$ suy biến. Phải thật sự có góc.
••• Vì sao saddle point chính xác hơn giao hai đường	Bốn hướng ràng buộc thay vì hai , và bốn hướng phân bố đều thay vì có thể chụm lại. Giao hai đường bị khuếch đại khi hai đường gần song song (tag chéo); saddle point không có cấu hình suy biến tương ứng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Hệ quả thiết kế: target lai	Tag vuông giới <i>nhận diện</i> (xa, bền, báo sai thấp, tự định danh); góc chessboard giới <i>đo</i> . Dùng cả hai theo giai đoạn: tag ngoại vi để bắt bảm, vùng ChArUco trung tâm để đo tinh.
●●● Trung bình N khung	$\sigma_N = \sigma / \sqrt{N}$. $N = 30 \Rightarrow 5,5$ lần, tốn 1 giây. Tỷ lệ lợi ích trên chi phí cao nhất trong chương, và tương thích với mọi phương pháp khác.
●●● Ba lý do trung bình không cho $1/\sqrt{N}$	Có thành phần <i>thiên lệch</i> (bảo hoà ở sàn khác không); nhiễu <i>tương quan</i> theo thời gian (đèn nhấp nháy, rung tuần hoàn — dốc nhỏ hơn 1/2 ngay từ đầu); <i>lượng tử hoá</i> ẩn trong thuật toán (giá trị trung bình rơi trên lưới đều).
●●● Biểu đồ chẩn đoán quan trọng nhất	σ_N theo N trên trục log-log. Dốc lý thuyết $-1/2$. Ba kiểu lệch khỏi nó ứng với ba nguyên nhân trên. Gần như miễn phí và nên vẽ trước mọi đầu tư khác.
●● Sản phẩm phụ đáng lấy	Mọi phương pháp sinh ra hiệp phương sai của góc miễn phí (nghịch đảo ma trận cấu trúc, hoặc nghịch đảo Hessian). Nó <i>dị hướng</i> và tính dị hướng mang thông tin thật.
●●● Hai lợi ích của việc dùng hiệp phương sai đúng	Nghiem chính xác hơn (góc tốt có tiếng nói lớn hơn); và — quan trọng hơn — <i>hiệp phương sai pose đầu ra trở nên trung thực</i> , mà cả chuỗi B/11, B/09 mục 7, B/13 mục 8 phụ thuộc vào đó.
●● Khớp quang trắc: vì sao ít dùng	Về nguyên tắc chính xác nhất (dùng mọi pixel chứ không chỉ quanh góc) nhưng chậm hơn một bậc, cần khởi tạo tốt, và nhảy với sai khác giữa mẫu và thực tế (bản, chiếu sáng). Nên dùng ở pha cuối khi robot đứng yên [6], [7].
●● Marker tròn: bước bắt buộc	Khớp ellipse rồi <i>hiệu chỉnh bias tâm bằng conic</i> [5]. Bỏ bước này thì có phép đo rất chính xác quanh một giá trị thiên lệch phụ thuộc tư thế — tệ hơn với ngân sách 5 mm.
●●● Các con số σ : cảnh báo	0,1–0,3 px cho AprilTag, 0,03–0,1 px cho ChArUco là <i>bậc độ lớn kinh nghiệm</i> không có điều kiện đo. Điều đáng tin là <i>thứ tự tương đối</i> (có cơ sở cấu trúc), không phải giá trị tuyệt đối.
●●● Thứ tự đầu tư để cải thiện σ	Trung bình nhiều khung $\rightarrow$ bật tinh chỉnh $\rightarrow$ saddle point/ChArUco $\rightarrow$ chiếu sáng $\rightarrow$ quang trắc $\rightarrow$ đổi camera. Và trước tất cả: vẽ σ_N theo N .
●● Khi nào chương này có đòn bẩy thấp	Khi ngân sách đang bị chi phối bởi các nguồn <i>hệ thống</i> (constellation chưa đo bằng BA, hiệu chuẩn kém). Cải thiện σ bốn lần trên một ngân sách như thế gần như không đổi gì.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV cornerSubPix	Cài (1.1); tham số winSize là đánh đổi ở mục 2	Ma trận cấu trúc đã tính sẵn bên trong — đáng sửa để lấy hiệp phương sai ra
OpenCV ChArUco	interpolateCornersCharuco dùng saddle point cho góc chessboard	Hiện thân của mục 3; nửa đo của target lai
deltille [3]	Saddle point trên lưới tam giác, tinh chỉnh bậc hai riêng	Lựa chọn khi cần độ chính xác góc cao nhất — B/12
ECC trong OpenCV [6]	findTransformECC — căn chỉnh quang trắc bất biến với thay đổi sáng	Dùng cho pha cuối khi đứng yên (B/13 mục 5)
Hiệu chỉnh conic [5]	Bù bias tâm cho target điểm tròn	Bắt buộc nếu dùng marker tròn; xem A/02 mục 6

MINI PROJECT · Vẽ đồ thị σ_N , và so ba cơ chế định vị góc trên cùng điều kiện · 2 ngày

Mục tiêu	trả lời hai câu hỏi quyết định chiến lược: hệ của tôi đang bị giới hạn bởi nhiễu hay bởi thiên lệch, và saddle point có thật sự chính xác hơn giao hai đường trên phần cứng của tôi không.
Dữ liệu	phần cứng thật. Một target lai tự chế: in một tấm có cả tag AprilTag, tag ArUco, và một vùng ChArUco cạnh nhau trên cùng một tấm nền cứng phẳng, để cả ba chịu cùng điều kiện quang học.
Các bước	phần một: cố định mọi thứ, chụp 1000 khung; với mỗi N trong $\{1, 2, 5, 10, 20, 50, 100, 200, 500\}$, chia 1000 khung thành các nhóm N khung, tính trung bình mỗi nhóm, rồi tính độ lệch chuẩn của các trung bình ấy; vẽ log-log và so với đường dốc $-1/2$. Phần hai: với cùng bộ ảnh, tính σ riêng cho góc AprilTag (khớp đường), góc ArUco có cornerSubPix, và góc ChArUco (saddle point); lập ở ba khoảng cách và ba góc nghiêng.
Xong khi	bạn có một đồ thị log-log và một bảng ba cột. Từ đồ thị: nếu nó bám đường dốc $-1/2$ tới $N = 500$ thì bạn đang bị giới hạn thuần bởi nhiễu và trung bình nhiều khung là đầu tư tốt nhất; nếu nó bão hoà ở $N = 20$ thì bạn có một thành phần thiên lệch và phải đi tìm nó trước mọi thứ khác. Từ bảng: tỉ số giữa ba cột là con số đo được của mệnh đề ở mục 3. Kiểm tra chéo bắt buộc: ba loại góc phải chịu cùng điều kiện quang học — nếu vùng ChArUco của bạn nằm ở rìa ảnh còn tag ở giữa thì so sánh vô nghĩa, vì bạn đang đo MTF chứ không đo thuật toán.
Nối với	C/15 nhận σ và con số bão hoà; B/13 mục 4 nhận số khung tối ưu cho stop-and-go.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Thứ tự là: trung bình nhiều khung (rẻ nhất, lợi nhất), rồi đổi thuật toán, rồi mới mua camera. Trung bình 30 khung cho $1/\sqrt{30} = 5,5$ lần cải thiện với chi phí một giây và không dòng code thuật toán nào. Chuyển từ khớp đường sang saddle point cho có lẽ hai tới ba lần, và đòi hỏi thiết kế target. Camera đắt hơn cho có lẽ hai lần và tốn tiền thật — và chỉ đáng nếu bạn đã kiểm tra rằng quang học là nút thắt chứ không phải nhiễu ảnh (A/06 mục 5B). Thứ tự này làm nhiều người bất ngờ vì lựa chọn tốt nhất không *cảm thấy* như kỹ thuật: nó không có thuật toán mới, không có paper để đọc, và nó nghe như gian lận. Nhưng nó là cách sử dụng thông tin đúng đắn, và nó tương thích với mọi phương pháp khác.
- A2. Con số đếm là *số hướng ràng buộc*: bốn thay vì hai. Góc tag vuông được xác định bằng cách lấy giao của hai đường thẳng, tức hai ràng buộc; và sai số của mỗi đường lan vào giao điểm với hệ số phụ thuộc góc giữa chúng — gần vuông góc thì hệ số cỡ 1, gần song song thì hệ số lớn tùy ý, và một tag nhìn chéo có hai cạnh kề gặp nhau ở góc khá nhỏ. Góc bàn cờ vua là một điểm yên ngựa: bốn góc phân tư quanh nó — hai sáng hai tối xen kẽ — mỗi cái đóng góp một ràng buộc, và bốn hướng ấy phân bố đều nên không có cấu hình nào làm hệ suy biến. Nói ngắn: hai hướng có thể chụm lại, bốn hướng phân bố đều thì không.
- A3. Tính chất: tại một góc thật, vectơ gradient của mọi pixel lân cận đều vuông góc với vectơ nối pixel đó tới góc. Lý do là gradient luôn vuông góc với cạnh (nó chỉ hướng độ sáng đổi nhanh nhất), và mọi cạnh tại một góc đều đi qua chính góc ấy [1]. Điều kiện để nó còn đúng: ma trận cấu trúc $\sum \nabla I \nabla I^T$ phải khả nghịch tốt, tức phải có gradient theo *hai* hướng khác nhau đáng kể trong cửa sổ. Một điểm nằm giữa một cạnh thẳng chỉ có gradient theo một hướng, ma trận suy biến, và phương pháp trượt tự do dọc cạnh. Điều kiện thứ hai, thực hành hơn: cửa sổ lân cận không được trùm sang cạnh khác hoặc sang nội dung khác, vì khi đó các gradient thêm vào không thoả tính chất trực giao với góc này.
- A4. Nguyên nhân một: có thành phần *thiên lệch* — méo còn sót, bias tâm ellipse, motion blur nếu còn chuyển động (A/06). Trung bình không chạm tới chúng. Nguyên nhân hai: nhiễu *tương quan theo thời gian* — đèn nhấp nháy, rung tuần hoàn của thân robot, nhiễu đọc có cấu trúc. Nguyên nhân ba: *lượng tử hoá* ẩn trong thuật toán, chẳng hạn nó luôn trả về bội của $1/8$ pixel do cách nội suy. Phép thử phân biệt cả ba bằng một biểu đồ: vẽ σ_N theo N trên trục log-log và so với đường dốc $-1/2$. Bão hoà ở một sàn khác không $\Rightarrow$ nguyên nhân một. Dốc nhỏ hơn $1/2$ ngay từ đầu $\Rightarrow$ nguyên nhân hai. Giá trị trung bình rơi đúng trên một lưới đều $\Rightarrow$ nguyên nhân ba.

- A5. Ba lý do. Nó chậm hơn một bậc vì nó tối ưu trên hàng nghìn pixel thay vì vài chục. Nó cần khởi tạo tốt nên vẫn phải chạy detector thường trước, tức nó là một bước *thêm* chứ không phải một bước *thay*. Và nó nhảy với sai khác giữa mẫu và thực tế: tag bị bắn, chiếu sáng không đều, hoặc ảnh mẫu không khớp chính xác với tag in ra — cả ba đều làm nó lệch, trong khi các phương pháp dựa trên cạnh thì bền hơn vì cạnh vẫn là cạnh dù độ sáng tuyệt đối đổi. Trường hợp nên dùng: khi robot *đã dừng* nên chậm không sao, khi tag sạch và chiếu sáng kiểm soát được, và khi bạn cần vết kiệt σ — tức chính xác pha cuối của chiến lược coarse-to-fine ở B/13 mục 5. Nói cách khác nó là công cụ chuyên dụng cho một giai đoạn, không phải lựa chọn mặc định.

NÓI LẠI THẬT ĐƠN GIẢN

Một câu hỏi tưởng như vô lý: làm sao đo được chính xác hơn cả kích thước một pixel?

Câu trả lời nằm ở chỗ pixel không phải một ô vuông chỉ biết đen hoặc trắng. Nó là một cái thùng đếm ánh sáng. Nếu một cái cạnh đen trắng đi qua giữa nó, nó đếm được lượng sáng ở giữa — và cái “ở giữa” ấy nói cho ta biết cạnh nằm gần bên nào. Pixel bên trái sáng tám phần, pixel bên phải sáng hai phần, vậy cạnh nằm lệch về bên phải. Thông tin đó có thật, và toàn bộ việc của chương này là dùng vớt nó đi.

Nhưng một cái cạnh chỉ nói được vị trí theo một chiều — chiều cắt ngang nó. Muốn biết một *điểm* nằm đâu thì cần ít nhất hai chiều, và đây là chỗ các phương pháp khác nhau.

Cách thứ nhất là lấy hai cạnh cắt nhau rồi tìm chỗ giao. Đây là cách của mấy cái tag vuông. Vấn đề là nếu hai cạnh gặp nhau ở một góc hẹp — chuyện xảy ra khi nhìn tag rất chếch — thì chỗ giao trượt rất xa chỉ vì một cạnh nghiêng đi chút xíu. Giống như hai cây gậy dài chạm đầu nhau: nghiêng một cây một chút thì đầu gậy trượt cả tấc.

Cách thứ hai là tìm một chỗ mà độ sáng đổi chiều theo *mọi* phía — như đỉnh của một cái yên ngựa, lên theo hướng này và xuống theo hướng kia. Góc của bàn cờ vua là một chỗ như thế: bốn ô, hai đen hai trắng, gặp nhau tại một điểm. Điểm ấy bị ghim từ bốn phía nên nó không trượt được, và đó là lý do bàn cờ đo chính xác hơn tag vuông.

Điều này dẫn tới một lời khuyên thực dụng: dùng cả hai loại. Tag vuông thì giỏi việc nhận ra từ xa và tự nói tên mình; bàn cờ thì giỏi việc đo cho chuẩn. Ghép chúng lại trên cùng một tấm và dùng mỗi cái cho việc nó giỏi.

Còn một mẹo cuối, đơn giản đến mức hay bị coi thường: *chụp nhiều tấm rồi lấy trung bình*. Ba mươi tấm thì sai số giảm hơn năm lần. Không cần thuật toán mới, không cần camera mới, chỉ cần robot đứng yên một giây. Nghe như gian lận, nhưng nó là cách dùng thông tin đúng đắn nhất trong cả chương.

Chỉ có một điều kiện, và nó quan trọng: cách này chỉ chữa được cái *run tay*, không chữa được cái *lệch ngắm*. Nếu ông kính làm mọi phép đo lệch về cùng một phía thì chụp một nghìn tấm vẫn lệch đúng bấy nhiêu. Cách kiểm rất đơn giản: vẽ xem sai số giảm thế nào khi tăng số tấm. Nếu nó cứ giảm đều thì bạn đang chỉ có run tay, cứ chụp thêm. Nếu nó giảm một lúc rồi dừng lại ở một mức nào đó thì bạn có lệch ngắm, và phải đi tìm nó thay vì chụp thêm.

Chỗ không rút gọn được: cái mức mà sai số dừng lại — cái sàn ấy — thì không đoán trước được. Nó là tổng của những thứ rất khác nhau: méo còn sót, ống kính mờ, tấm marker hơi cong. Phải đo mới biết, và biểu đồ “sai số theo số tấm chụp” là cách rõ nhất để biết mình đang ở đâu.

Một câu: *Chụp nhiều tấm chữa được tay run, không chữa được mắt lệch — và cái biểu đồ đơn giản nhất cho bạn biết mình đang mắc bệnh nào.*

1.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *saddle point chính xác hơn giao hai đường vì bốn hướng ràng buộc thay vì hai* — bị bác bỏ nếu mini project cho thấy σ của góc ChArUco và σ của góc AprilTag tương đương trên cùng điều kiện quang học. Tôi dẫn nó bằng lập luận cấu trúc (cửa a) và nó được ủng hộ gián tiếp bởi thực tế rằng cộng đồng hiệu chuẩn dùng chessboard; nhưng nó *chưa* qua cửa (b) hay (c) với một so sánh định lượng. Đây là mệnh đề nền của đề xuất target lai và là nợ kiểm chứng lớn nhất của chương.

Mệnh đề thứ hai — *trung bình N khung cho $1/\sqrt{N}$* — là định lý giới hạn trung tâm và nó đúng dưới giả thiết. Điều đáng kiểm không phải mệnh đề mà là *giả thiết*: nhiều có thật sự độc lập giữa các khung trên hệ của bạn không. Mini project phần một là phép kiểm ấy, và tôi dự đoán rằng nhiều hệ sẽ thấy bão hoà sớm hơn mong đợi — nếu vậy thì đó là một phát hiện có giá trị vì nó chỉ ra một nguồn thiên lệch cần tìm.

Mệnh đề thứ ba — *cải thiện σ có đòn bẩy thấp khi ngân sách bị chi phối bởi nguồn hệ thống* — bị bác bỏ nếu trong một hệ thực, cải thiện σ bốn lần cho cải thiện sai số cuối cùng gần bốn lần. Nếu vậy thì hệ ấy đã xử lý tốt các nguồn hệ thống, và lời khuyên về thứ tự đầu tư không áp dụng cho nó.

Cuối cùng, tôi đã cố ý không phát biểu “phương pháp X cho $\sigma = Y$ px”, và mục 6 giải thích vì sao: các con số lưu hành không có điều kiện đo, và trích chúng như kết quả đo là vi phạm chuẩn của kho.

1.11 Kết nối

Chương này cấp một con số duy nhất cho phần còn lại của cây, và con số ấy là σ .

Nối lên trên. A/05-lan-truyen-sai-so-pixel-sang-pose là chương tiêu thụ trực tiếp: σ là tử số của cả bốn công thức ở đó, và nó là đại lượng duy nhất trong bốn công thức mà phần mềm ảnh hưởng được. A/06-hieu-ung-vat-ly-va-quang-hoc mục 5 đặt trần cứng cho σ , và mục 7 ở đó giải thích vì sao trung bình nhiều khung không chạm tới thiên lệch — nền của mục 5A ở đây. A/02-homography-va-hinh-hoc-phang mục 6 cấp cảnh báo về bias tâm ellipse mà mục 4 ở đây nhắc lại.

Nối ngang trong Phần B. B/07-detection-pipeline là chương tiền nhiệm trực tiếp: nó mô tả cơ chế định vị góc mặc định, chương này bàn cách cải thiện. Hai chương nên đọc liền. B/09-multi-marker-constellation là chương có đòn bẩy cao hơn, và mục 6 ở đây giải thích vì sao: cải thiện σ không chạm tới các dòng hệ thống mà B/09 mục 5 xử lý. B/11-uoc-luong-theo-thoi-gian mở rộng mục 5 từ trung bình đơn giản sang hợp nhất có trọng số theo thời gian, và nó phụ thuộc vào hiệp phương sai mà mục 7 ở đây cung cấp. B/13-docking-control mục 4 là chỗ trung bình nhiều khung được tích hợp vào chiến lược điều khiển — stop-and-go không chỉ để xoá thiên lệch mà còn để tạo điều kiện cho trung bình.

Nối xuống Phần C. C/15-ngan-sach-sai-so nhận σ làm một trong bốn đầu vào, và nó nhận từ mục 5A một cảnh báo phương pháp luận: khi lập ngân sách, phải dùng σ sau khi trung bình nếu chiến lược có trung bình, và phải kiểm rằng trung bình thật sự hoạt động bằng đồ thị log-log. C/16-danh-gia-va-nghiem-thu nhận mini project của chương này làm một mục trong quy trình nghiệm thu.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao độ chính xác dưới pixel là khả thi về mặt vật lý, và thông tin nằm ở đâu?
2. cornerSubPix dựa trên một tính chất hình học. Phát biểu nó, và nêu chính xác cấu hình làm nó suy biến.
3. Saddle point chính xác hơn giao hai đường vì một lý do đếm được. Nêu lý do ấy, và nêu cấu hình cụ thể làm giao hai đường tệ nhất.
4. Bạn vẽ σ_N theo N và thấy nó bão hoà ở $N = 20$. Nêu kết luận, và nêu việc tiếp theo nên làm — và việc nào *không* nên làm.
5. Mọi phương pháp trong chương sinh ra một đại lượng miễn phí mà hầu hết cài đặt vứt đi. Đại lượng đó là gì, vì sao tính dị hướng của nó quan trọng, và hai lợi ích của việc dùng nó?
6. Vì sao các con số σ lưu hành trong tài liệu không dùng được để nghiệm thu, và điều gì trong chúng thì vẫn đáng tin?

7. Trong trường hợp nào thì cải thiện σ bốn lần gần như không cải thiện sai số cuối cùng chút nào?

Nguồn

- [1] W. Förstner and E. Gülch, “A Fast Operator for Detection and Precise Location of Distinct Points, Corners and Centres of Circular Features,” *ISPRS Intercommission Workshop*, 1987, mức tin A; nguồn của phép tính chỉnh góc bằng tính trực giao gradient, tức cornerSubPix.
- [2] L. Lucchese and S. K. Mitra, “Using Saddle Points for Subpixel Feature Detection in Camera Calibration Targets,” *Asia-Pacific Conference on Circuits and Systems*, 2002, mức tin B; khớp bậc hai quanh điểm yên ngựa.
- [3] H. Ha, M. Perdoch, H. Alismail, et al., “Deltile Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tính chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [4] S. Ghosal and R. Mehrotra, “Orthogonal Moment Operators for Subpixel Edge Detection,” *Pattern Recognition*, vol. 26, no. 2, pp. 295–306, 1993, mức tin B; định vị cạnh dưới pixel bằng moment Zernike.
- [5] J. Heikkilä, “Geometric Camera Calibration Using Circular Control Points,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1066–1077, 2000, mức tin A; nguồn chuẩn cho bias tâm ellipse và cách hiệu chỉnh bằng conic.
- [6] G. D. Evangelidis and E. Z. Psarakis, “Parametric Image Alignment Using Enhanced Correlation Coefficient Maximization,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 10, pp. 1858–1865, 2008, mức tin A; ECC, căn chỉnh quang trắc bất biến với thay đổi sáng.
- [7] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; inverse compositional alignment.
- [8] J. Wang and E. Olson, “AprilTag 2: Efficient and Robust Fiducial Detection,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [9] M. Kalaitzakis, B. Cain, S. Carroll, et al., “Fiducial Markers for Pose Estimation: Overview, Applications and Experimental Comparison of the ARTag, AprilTag, ArUco and STag Markers,” in *Journal of Intelligent & Robotic Systems*, mức tin B; so sánh thực nghiệm nhiều họ tag — nguồn duy nhất trong cây có số đo trực tiếp so sánh giữa các họ, vol. 101, 2021.